EXPERIMENT NO: 8

ROLL No:

NAME:

TITLE: Write a Program to Create Multiple Tasks and understand the Multitasking capabilities of RTOS(FreeRTOS).

Apparatus: Arduino Board, Computer,FreeRTOS Source Code, Arduino IDE, LED, Resistor, Virtual Terminal, potentiometer and Proteus software.

Theory:
Real time operating system popularly known as RTOS provides controller with the ability to respond to input and complete tasks within a specific period of time based on priority. On the first look, an RTOS might sound like just any other embedded program or firmware, but it is built on the architecture of an Operating system. Hence, like any operating system RTOS can allow multiple programs to execute at the same time supporting **multiplexing**. As we know the core of a processor or controller can only execute a single instruction at a time, but the RTOS has something called the **scheduler** which decides which instruction to execute first and thus executes the instructions of multiple programs one after the other. Technically an RTOS only creates an illusion of multi-taking by executing paralleled instructions one at a time.

Arduino code of simulteniously running three task
1. Blink led with 200ms delay
2. Blink led with 300ms delay
3. Print counter variable on serial monitor

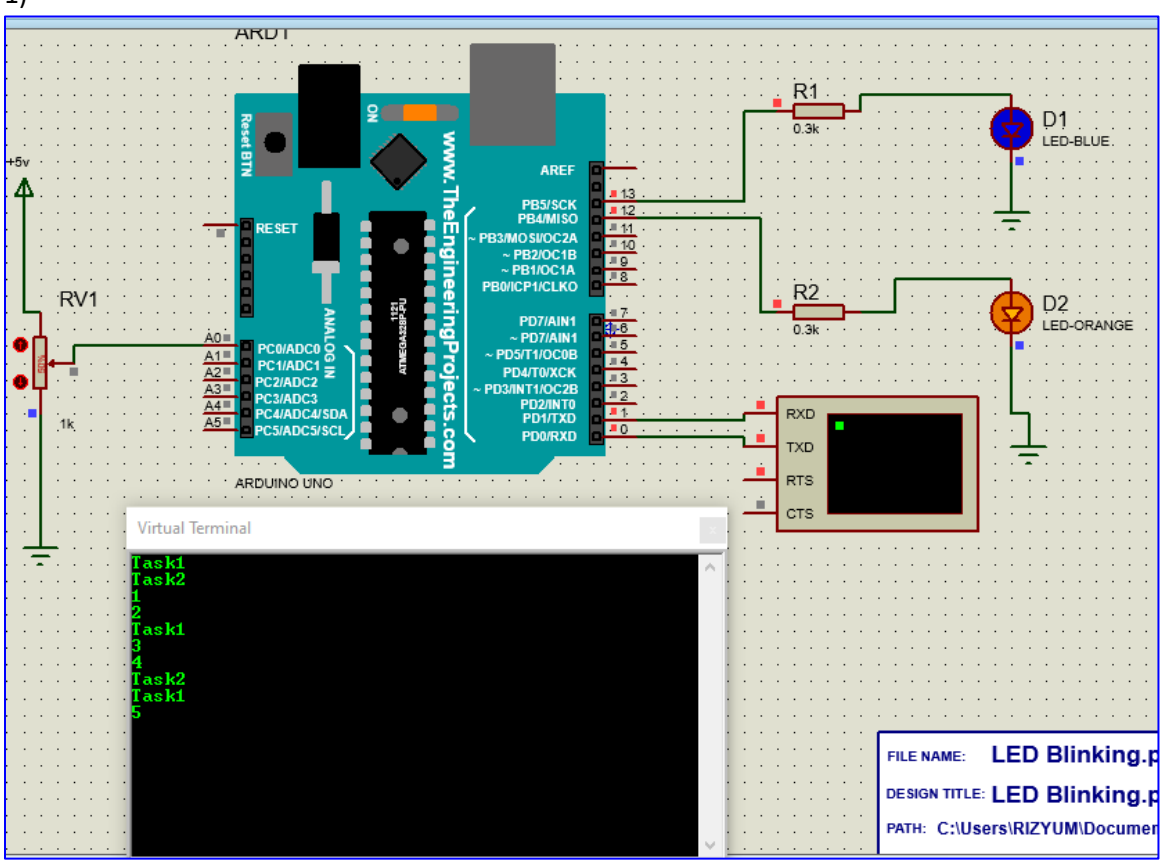
Code:
```
#include <Arduino_FreeRTOS.h>

// Define the pins for the LEDs
#define LED1 13
#define LED2 12

// Task handles
TaskHandle_t Task1_Handle;
TaskHandle_t Task2_Handle;
TaskHandle_t Task3_Handle;

// Task functions
```

```
void Task1(void *pvParameters);
void Task2(void *pvParameters);
void Task3(void *pvParameters);

void setup() {
  // Initialize serial communication
  Serial.begin(9600);

  // Set the LED pins as outputs
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);


  // Create Task 1
  xTaskCreate(Task1, "Task1", 128, NULL, 1, &Task1_Handle);

  // Create Task 2
  xTaskCreate(Task2, "Task2", 128, NULL, 1, &Task2_Handle);
  xTaskCreate(Task3, "Task3", 128, NULL, 1, &Task3_Handle);

  // Start the scheduler
  vTaskStartScheduler();
}

void loop() {
  // Empty loop since tasks are running in FreeRTOS scheduler
}

// Task 1: Blinks LED1 at a rate of 500ms
void Task1(void *pvParameters) {
  (void) pvParameters;

  while (1) {
    Serial.println("Task1");
    digitalWrite(LED1, HIGH);
    vTaskDelay(500 / portTICK_PERIOD_MS);
    digitalWrite(LED1, LOW);
    vTaskDelay(500 / portTICK_PERIOD_MS);
  }
}

// Task 2: Blinks LED2 at a rate of 1000ms
void Task2(void *pvParameters) {
  (void) pvParameters;
```

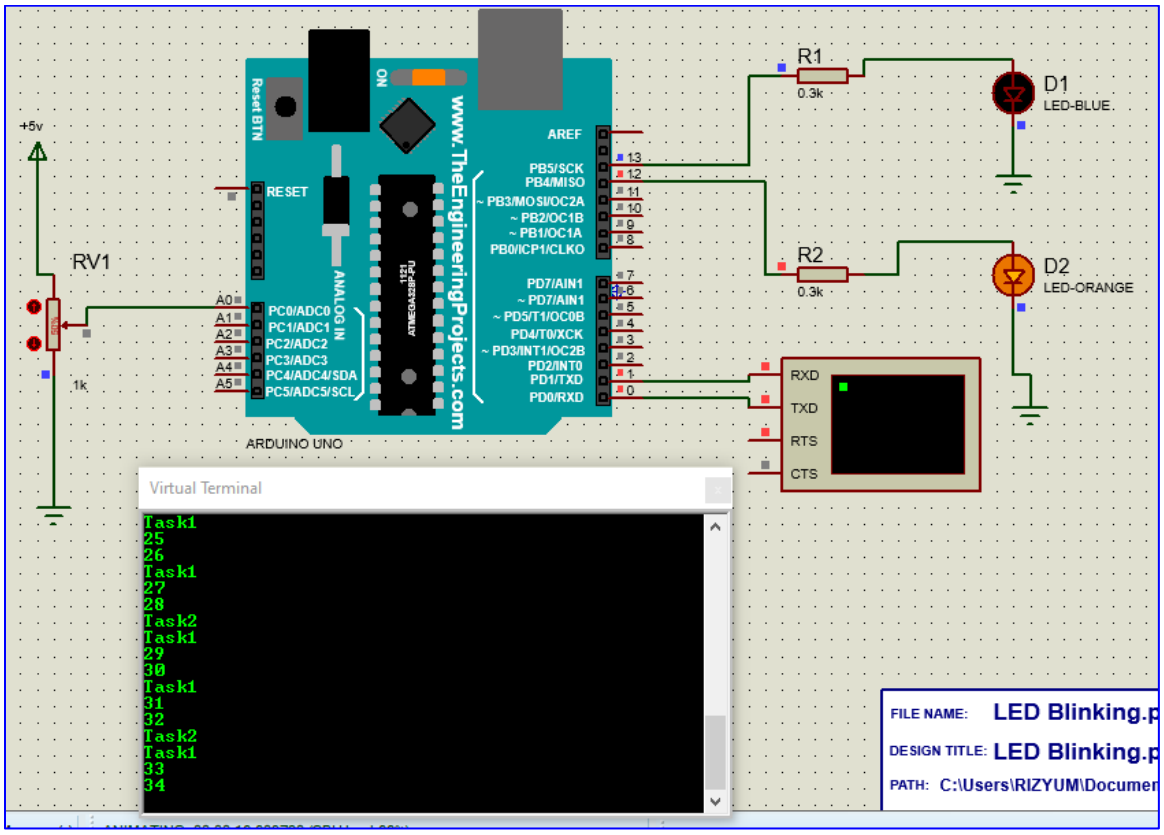
```
  while (1) {
    Serial.println("Task2");
    digitalWrite(LED2, HIGH);
    vTaskDelay(1000 / portTICK_PERIOD_MS);
    digitalWrite(LED2, LOW);
    vTaskDelay(1000 / portTICK_PERIOD_MS);
  }
}
void Task3(void *pvParameters) {
  int counter = 0;
  (void) pvParameters;

  while (1) {
    counter++;
    Serial.println(counter);
  vTaskDelay(500 / portTICK_PERIOD_MS);
  }
}
```

Result:

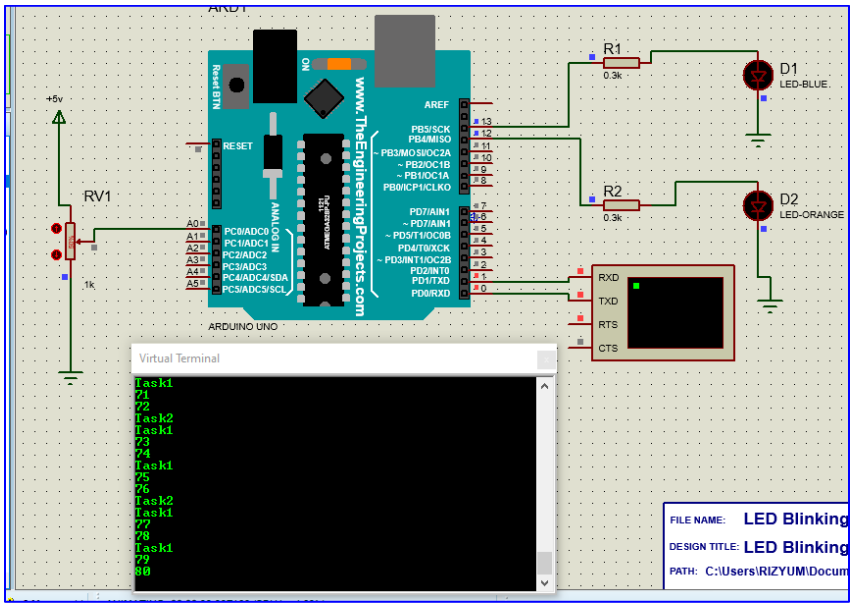
1)



2)


R1
0.3k
D1
LED-BLUE
R2
0.3k
D2
LED-ORANGE
+5v
RV1
1k
RESET
AREF
PB5/SCK
PB4/MISO
~ PB3/MOSI/OC2A
~ PB2/OC1B
~ PB1/OC1A
PB0/ICP1/CLKO
PD7/AIN1
~ PD7/AIN1
~ PD5/T1/OC0B
PD4/T0/XCK
~ PD3/INT1/OC2B
PD2/INT0
PD1/TXD
PD0/RXD
A0
A1
A2
A3
A4
A5
PC0/ADC0
PC1/ADC1
PC2/ADC2
PC3/ADC3
PC4/ADC4/SDA
PC5/ADC5/SCL
ANALOG IN
ARDUINO UNO
www.TheEngineeringProjects.com
RXD
TXD
RTS
CTS
Virtual Terminal
Task1
25
26
Task1
27
28
Task2
Task1
29
30
Task1
31
32
Task2
Task1
33
34
FILE NAME: LED Blinking.p
DESIGN TITLE: LED Blinking.p
PATH: C:\Users\RIZYUM\Documen

3)


R1
0.3k
D1
LED-BLUE
R2
0.3k
D2
LED-ORANGE
+5v
RV1
1k
RESET
AREF
ARDUINO UNO
www.TheEngineeringProjects.com
RXD
TXD
RTS
CTS
Virtual Terminal
Task1
71
72
Task2
Task1
73
74
Task1
75
76
Task2
Task1
77
78
Task1
79
80
FILE NAME: LED Blinking
DESIGN TITLE: LED Blinking
PATH: C:\Users\RIZYUM\Docum

Conclusion: The program demonstrates the creation of multiple tasks using FreeRTOS. Each task runs independently with its own function and executes in a multitasking environment provided by the Real-Time Operating System (RTOS). This allows for efficient utilization of system resources and enables tasks to execute concurrently without interfering with each other. FreeRTOS provides mechanisms like task scheduling and prioritization to manage task execution, ensuring real-time responsiveness and determinism in embedded systems applications.